

Module 12 – Part 2 | Observables & RxJS

Master asynchronous data streams — the engine powering every Angular HTTP call, form event, and real-time update.

Observable

Observer

Cold vs Hot

next / error / complete

fromEvent

interval

map · filter · switchMap

catchError · retry

1. Why Do We Need Observables?

Before writing a single line of RxJS, it is important to understand the problem it is solving.

In an Angular application, data rarely arrives all at once. A user types into a search box — each keystroke is a new piece of data. A REST API returns a response — but only after some time. A WebSocket pushes stock prices every second. All of these are **asynchronous streams of data**.

JavaScript has handled async code in different ways over the years:

Approach	What it does	Limitation
Callback	Run a function when something finishes	Nests deeply — "callback hell"
Promise	Wraps a single future value	One value only, cannot be cancelled, no operators
Observable	Wraps a <i>stream</i> of values over time	Slightly more to learn upfront, but very powerful

Analogy

Promise vs Observable — The Pizza Shop Analogy.

A **Promise** is like ordering one pizza. You wait. It arrives. Done.

An **Observable** is like a pizza subscription. Every Friday a pizza arrives, until you cancel. You can also choose to only receive vegetarian pizzas (filter), or swap your order (switchMap). That flexibility is what Observables bring.

RxJS (Reactive Extensions for JavaScript) is the library Angular uses to work with Observables. It provides the `Observable` class and a large set of **operators** — functions that transform, filter, and combine streams.

2. Observable & Observer — The Two Roles

Every RxJS pattern involves exactly two participants:

OBSERVABLE

The data source.
It produces values
over time.

think of it as a
"radio station"

OBSERVER

The consumer.
It reacts to those values
by defining three callbacks.

think of it as a
"radio listener"

Observable	emits	→	Observer
	values		<code>.next(value)</code>
	errors		<code>.error(err)</code>
	completion		<code>.complete()</code>

- The **Observable** defines *what* data to produce and *when*.
- The **Observer** defines *what to do* when that data arrives.
- The connection between them is called a **subscription**.

Note

Nothing happens until someone **subscribes**. An Observable is lazy — it does not run its logic until you call `.subscribe()`.

STEP 1 Your First Observable — Subscribing to an HTTP Call

In Module 12-1, we called `http.get()`. That method returns an **Observable**. Here is the simplest way to read its value:

```
// product.service.ts
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ProductService {
  constructor(private http: HttpClient) {}

  getProducts() {
    // http.get() returns an Observable – nothing happens yet
    return this.http.get('https://fakestoreapi.com/products');
  }
}
```

```
// product.component.ts
import { Component, OnInit } from '@angular/core';
import { ProductService } from './product.service';

@Component({ selector: 'app-products', templateUrl: './products.component.html' })
export class ProductsComponent implements OnInit {
  products: any[] = [];

  constructor(private productService: ProductService) {}

  ngOnInit() {
    // .subscribe() is the trigger – NOW the HTTP request fires
    this.productService.getProducts().subscribe({
      next: (data: any) => {
        this.products = data; // runs when data arrives
        console.log('Products:', data);
      },
      error: (err) => {

```

```

    console.error('Error:', err); // runs if the request fails
  },
  complete: () => {
    console.log('Stream finished'); // runs when no more data will come
  }
});
}
}

```

Output:

```

Products: [ {id:1, title:"Fjallraven Backpack", price:109.95, ...}, {id:2, ...}, ... ]
Stream finished

```

The object passed to `.subscribe()` is the **Observer**. It contains three optional callback functions that handle the three things an Observable can do.

3. The Three Callback Methods: `next()` , `error()` , `complete()`

Callback	When it runs	Receives	Can run multiple times?
<code>next(value)</code>	Each time the Observable emits a new value	The emitted value	 Yes — as many times as needed
<code>error(err)</code>	When something goes wrong	The error object	No — Observable stops after this
<code>complete()</code>	When the Observable signals it has no more values	Nothing	No — exactly once, then stream ends

Analogy

Think of a water tap.

`next()` = each drop of water that flows out.

`error()` = the pipe bursts — water stops, something went wrong.

`complete()` = you turn the tap off properly — no more water, but everything is fine.

⚠ Important

After either `error()` or `complete()` fires, the Observable will **never** call `next()` again. These two are terminal events.

4. Cold vs Hot Observables

This is one of the most common interview topics in RxJS. The distinction is simple:

	Cold Observable	Hot Observable
Data starts	Only when someone subscribes	Regardless of subscribers — already running
Each subscriber	Gets its own independent stream from the beginning	Joins the same shared stream, mid-way
Real examples	HTTP request, custom Observable	Mouse clicks, WebSocket, <code>Subject</code>
Analogy	A Netflix movie — you press play, it starts from the beginning just for you	A live TV broadcast — you tune in and see what is happening right now

COLD OBSERVABLE

Subscriber A subscribes → Observable starts → A gets: 1, 2, 3

Subscriber B subscribes → Observable starts → B gets: 1, 2, 3

(each subscriber gets a fresh, independent stream)

HOT OBSERVABLE

Observable is already running: ... 1, 2, 3, 4, 5, 6 ...

Subscriber A joins at 3 → A gets: 3, 4, 5, 6

Subscriber B joins at 5 → B gets: 5, 6

(both share the same stream)

Note

In most Angular work you deal with **Cold Observables** (HTTP calls). Hot Observables appear when you learn `Subject` — which is covered in the services and state management modules.

5. Creating Observables

So far we have subscribed to Observables that Angular gives us. Now we create our own. This builds a deep understanding of how they work internally.

STEP 2 Creating an Observable from Scratch

Use the `new Observable()` constructor. You receive an **observer** object as an argument and use it to emit values manually:

```
import { Observable } from 'rxjs';

// This Observable emits three numbers, then completes
const numbers$ = new Observable((observer) => {
  observer.next(10);      // emit value 1
  observer.next(20);      // emit value 2
  observer.next(30);      // emit value 3
  observer.complete();    // signal: no more values
});

// Nothing runs until we subscribe
numbers$.subscribe({
  next: (val) => console.log('Received:', val),
  error: (err) => console.error('Error:', err),
  complete: () => console.log('Done!')
});
```

Output:

Received: 10

Received: 20

Received: 30

Done!

Note

The `$` suffix on `numbers$` is a widely used naming convention to indicate a variable holds an Observable. It is optional but strongly recommended — it makes code much easier to read.

STEP 3

fromEvent — Turning DOM Events into Observables

Instead of using `addEventListener`, RxJS lets you wrap any DOM event into an Observable using `fromEvent`. This is a **Hot Observable** — the event source (the button) exists independently of your subscription.

```
// component.ts
import { Component, OnInit, OnDestroy, ElementRef, ViewChild } from '@angular/core';
import { fromEvent, Subscription } from 'rxjs';

@Component({
  selector: 'app-click-demo',
  template: `<button #myBtn>Click Me</button>
    <p>Click count: {{ count }}</p>`
})
export class ClickDemoComponent implements OnInit, OnDestroy {
  @ViewChild('myBtn', { static: true }) myBtn!: ElementRef;

  count = 0;
  private clickSub!: Subscription;

  ngOnInit() {
    // fromEvent wraps the button's click event as an Observable
    const click$ = fromEvent(this.myBtn.nativeElement, 'click');

    this.clickSub = click$.subscribe({
      next: (event) => {
        this.count++;
        console.log('Button clicked!', event);
      }
    });
  }
}
```

```

    }
  });
}

ngOnDestroy() {
  // ALWAYS unsubscribe to avoid memory leaks
  this.clickSub.unsubscribe();
}
}

```

Output:

[After clicking the button 3 times]

```

Button clicked! MouseEvent {...}
Button clicked! MouseEvent {...}
Button clicked! MouseEvent {...}
Click count: 3

```

⚠ Important

Always unsubscribe in `ngOnDestroy`. If you do not unsubscribe from a `fromEvent` Observable, the event listener keeps running even after the component is destroyed — this is a **memory leak**.

STEP 4 `interval` — Emitting Values on a Timer

`interval(n)` emits an incrementing number every `n` milliseconds. It never completes on its own — you must unsubscribe when done.

```

import { Component, OnInit, OnDestroy } from '@angular/core';
import { interval, Subscription } from 'rxjs';

@Component({
  selector: 'app-timer',
  template: `<p>Tick: {{ tick }}</p>
    <button (click)="stopTimer()">Stop</button>`
})
export class TimerComponent implements OnInit, OnDestroy {

```



```
tick = 0;
private timer$!: Subscription;

ngOnInit() {
  // Emits 0, 1, 2, 3 ... every 1 second
  this.timer$ = interval(1000).subscribe({
    next: (count) => {
      this.tick = count;
      console.log('Tick:', count);
    }
  });
}

stopTimer() {
  this.timer$.unsubscribe(); // manually stop
  console.log('Timer stopped');
}

ngOnDestroy() {
  this.timer$.unsubscribe(); // also stop if component is destroyed
}
}
```

Output:

```
Tick: 0
Tick: 1
Tick: 2
... [every 1 second until Stop is clicked]
Timer stopped
```

Note

`interval` is commonly used for polling — checking an API every few seconds for new data (e.g., a live cricket score board, a stock price ticker).

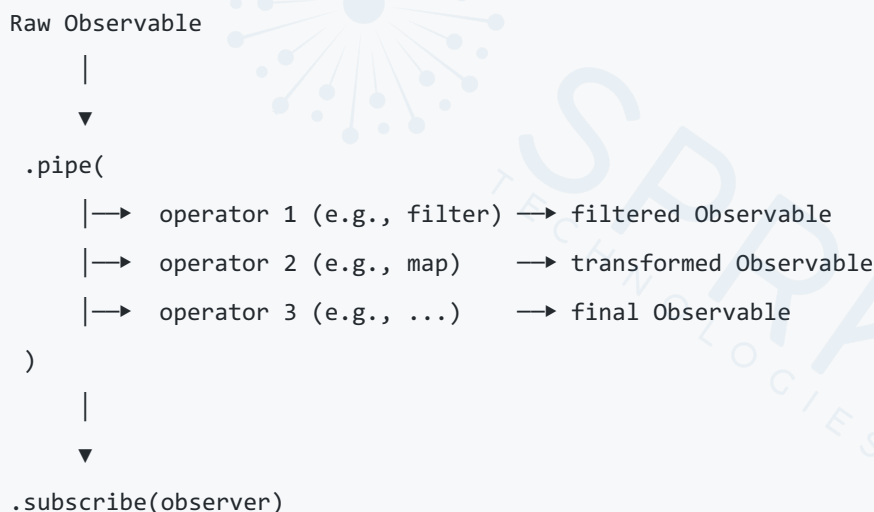
6. RxJS Operators — Transforming the Stream

Operators are **pure functions** that take an Observable as input and return a new Observable as output. They never modify the original stream — they create a transformed copy of it. You chain them together using the `pipe()` method.

Analogy

Think of a water treatment plant.

Raw water (Observable) enters a series of filters and treatment stages (operators). Each stage transforms the water without changing what went in. The clean water at the end is a new, processed stream — ready to use.



```
Raw Observable
  |
  ▼
.pipe(
  |→ operator 1 (e.g., filter) → filtered Observable
  |→ operator 2 (e.g., map)    → transformed Observable
  |→ operator 3 (e.g., ...)    → final Observable
)
  |
  ▼
.subscribe(observer)
```

The diagram illustrates the RxJS `pipe()` method. It starts with a 'Raw Observable' at the top. A vertical line with a downward arrow leads to the `.pipe()` method. Inside `.pipe()`, three operators are listed, each followed by an arrow pointing to its output: 'operator 1 (e.g., filter)' leads to 'filtered Observable', 'operator 2 (e.g., map)' leads to 'transformed Observable', and 'operator 3 (e.g., ...)' leads to 'final Observable'. The `.pipe()` block is enclosed in parentheses. Below the parentheses, another vertical line with a downward arrow leads to the `.subscribe(observer)` method at the bottom.

STEP 5 `map` — Transform Each Value

`map` works exactly like `Array.map` — it runs a function on every emitted value and emits the result. Use it whenever you need to reshape or extract data from a stream.

```
import { Component, OnInit } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { map } from 'rxjs/operators';
```

```
@Component({
  selector: 'app-products',
  template: `
    <ul>
      <li *ngFor="let name of productNames">{{ name }}</li>
    </ul>`
})
export class ProductsComponent implements OnInit {
  productNames: string[] = [];

  constructor(private http: HttpClient) {}

  ngOnInit() {
    this.http.get<any[]>('https://fakestoreapi.com/products')
      .pipe(
        // The API returns full product objects.
        // We only want the title of each product.
        map((products) => products.map(p => p.title))
      )
      .subscribe({
        next: (names) => {
          this.productNames = names;
          console.log('Names only:', names);
        }
      });
  }
}
```

Output:

```
Names only: ["Fjallraven Backpack", "Mens Casual Premium Slim Fit T-Shirts", ...]
```

Rendered list:

- Fjallraven Backpack
- Mens Casual Premium Slim Fit T-Shirts
- Mens Cotton Jacket
- ...

STEP 6 `filter` — Pass Only Matching Values

Now let us combine `filter` with `map`. `filter` only allows values through that pass a condition — like `Array.filter`. In the example below we fetch products but only show those priced below \$50, and we only display their titles.

```
import { map, filter } from 'rxjs/operators';

ngOnInit() {
  this.http.get<any[]>('https://fakestoreapi.com/products')
    .pipe(
      // Step 1 – keep only products under $50
      map((products) => products.filter(p => p.price < 50)),

      // Step 2 – extract only the title from each remaining product
      map((affordable) => affordable.map(p => p.title))
    )
    .subscribe({
      next: (names) => {
        this.productNames = names;
        console.log('Affordable products:', names);
      }
    });
}
```

Output:

```
Affordable products: ["Mens Cotton Jacket", "Mens Casual Slim Fit", ...]
```

Note

You can also use the RxJS `filter` operator from `rxjs/operators` directly on scalar streams. In the example above we used JavaScript's `Array.filter` inside a `map`, which is common and perfectly valid when operating on an array emitted as a single value.

STEP 7
switchMap — Switching to a New Observable

`switchMap` is used when the output of one Observable must become the *input* of another Observable. The most common use case: the user types in a search box, and for each keystroke you want to trigger a new HTTP request.

The key behaviour of `switchMap` : if a new value arrives *before* the previous inner Observable has completed, it **cancels the previous one** and starts the new one. This prevents stale responses from appearing on screen.

User types "a" → HTTP GET /search?q=a —(cancelled, "an" typed)→ ✗
 User types "an" → HTTP GET /search?q=an —(cancelled, "ang" typed)→ ✗
 User types "ang" → HTTP GET /search?q=ang → ✓ results shown

```
import { Component, OnInit } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { FormControl } from '@angular/forms';
import { switchMap, debounceTime } from 'rxjs/operators';

@Component({
  selector: 'app-search',
  template: `
    <input [formControl]="searchControl" placeholder="Search product..." />
    <ul>
      <li *ngFor="let item of results">{{ item.title }}</li>
    </ul>`
})
export class SearchComponent implements OnInit {
  searchControl = new FormControl('');
  results: any[] = [];

  constructor(private http: HttpClient) {}

  ngOnInit() {
    this.searchControl.valueChanges // Observable – emits on every keystroke
      .pipe(
        debounceTime(300),           // wait 300ms after the user stops typing
        switchMap((query) =>
          // For every new search term, cancel the old request and start a new one
        )
      )
      .subscribe(results);
  }
}
```

```
        this.http.get<any[]>(`https://fakestoreapi.com/products?title=${query}`)
      )
    )
    .subscribe({
      next: (data) => this.results = data
    });
  }
}
```

Output:

[User types "jacket" slowly]

→ Previous requests cancelled automatically

→ Final request fires after user pauses

List updates: Mens Cotton Jacket, ...

⚠ Important

`switchMap` cancels the previous inner Observable. If you do *not* want cancellation (e.g., saving multiple records in parallel), use `mergeMap` instead. For interview purposes: know that `switchMap` cancels, `mergeMap` does not.

STEP 8

catchError — Handling Errors in the Stream

When an HTTP request fails, the Observable emits an error. If you do not handle it, your stream terminates and the UI breaks silently. `catchError` intercepts the error and lets you return a safe fallback value — keeping the stream alive or ending it gracefully.

```
import { HttpClient } from '@angular/common/http';
import { catchError } from 'rxjs/operators';
import { of } from 'rxjs'; // 'of' creates an Observable from a plain value

@Injectable({ providedIn: 'root' })
export class ProductService {
  constructor(private http: HttpClient) {}

  getProducts() {
```

```
return this.http.get<any[]>('https://fakestoreapi.com/products')
    .pipe(
      catchError((error) => {
        // Log the error for debugging
        console.error('Request failed:', error.message);

        // Return an empty array so the component does not crash
        return of([]);
      })
    );
}
```

```
// component – subscribes as normal, no need to handle error separately
ngOnInit() {
  this.productService.getProducts().subscribe({
    next: (data) => this.products = data // gets [] if the API failed
  });
}
```

Output:

[If the API is unreachable]

```
Request failed: Http failure response for https://fakestoreapi.com/products: 0 Unknown Error
this.products = [] (empty list renders, no crash)
```

Note

`of(value)` creates a simple Observable that emits one value and immediately completes. It is a convenience function used often in error recovery — returning a safe default.

STEP 9 **retry** — Automatically Retrying a Failed Request

Network requests sometimes fail temporarily — a server blip, a mobile network hiccup. `retry(n)` automatically re-subscribes to the Observable up to `n` times before giving up. Place it *before* `catchError` so retries happen first, and only if all retries fail does the error handler run.

```
import { retry, catchError } from 'rxjs/operators';
import { of } from 'rxjs';

getProducts() {
  return this.http.get<any[]>('https://fakestoreapi.com/products')
    .pipe(
      retry(2), // try up to 2 more times if the first attempt fails
      catchError((error) => {
        console.error('All retries failed:', error.message);
        return of([]); // return empty array as fallback
      })
    );
}
```

Output:

[Simulating a network failure]

Attempt 1 → FAILED

Attempt 2 (retry 1) → FAILED

Attempt 3 (retry 2) → FAILED

All retries failed: Http failure response: 0 Unknown Error

this.products = []

Without retry:

→ Request 1 fails → catchError → []

With retry(2):

→ Request 1 fails

→ (auto) Request 2 fails

→ (auto) Request 3 fails

→ catchError → []

7. Complete Example — Operators Chained Together

The following example demonstrates a realistic service that combines `map`, `retry`, and `catchError` in a single pipe, along with a component that uses the service:

```
// product.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { map, retry, catchError } from 'rxjs/operators';
import { of } from 'rxjs';

export interface Product {
  id: number;
  title: string;
  price: number;
  category: string;
}

@Injectable({ providedIn: 'root' })
export class ProductService {
  private apiUrl = 'https://fakestoreapi.com/products';

  constructor(private http: HttpClient) {}

  getAffordableProducts(maxPrice: number) {
    return this.http.get<Product[]>(this.apiUrl).pipe(
      // 1. Keep only products below the given price
      map((products) => products.filter(p => p.price <= maxPrice)),

      // 2. Sort them cheapest first
      map((filtered) => filtered.sort((a, b) => a.price - b.price)),

      // 3. Retry twice on network failure
      retry(2),

      // 4. Return empty array if all retries fail
      catchError((err) => {
        console.error('Failed to load products:', err.message);
        return of([] as Product[]);
      })
    );
  }
}
```

```
// product-list.component.ts
import { Component, OnInit } from '@angular/core';
import { Product, ProductService } from '../product.service';

@Component({
  selector: 'app-product-list',
  template: `
    <h2>Products under $50</h2>
    <p *ngIf="products.length === 0">No products found or failed to load.</p>
    <ul>
      <li *ngFor="let p of products">
        {{ p.title }} — ${p.price}
      </li>
    </ul>`
})
export class ProductListComponent implements OnInit {
  products: Product[] = [];

  constructor(private productService: ProductService) {}

  ngOnInit() {
    this.productService.getAffordableProducts(50).subscribe({
      next: (data) => this.products = data,
      complete: () => console.log('Product stream complete')
    });
  }
}
```

Output:

Products under \$50

- Mens Cotton Jacket — \$55.99 (filtered out)
- Mens Casual Slim Fit — \$15.99
- Women's Silk — \$29.95
- White Gold Plated Bracelet — \$6.95

Product stream complete

8. Memory Leaks — Why You Must Unsubscribe

HTTP Observables complete automatically after emitting one response — so you do not need to manually unsubscribe from them. But Observables like `interval` or `fromEvent` **never complete**. If you subscribe to these and never unsubscribe, they keep running in memory even after the component is destroyed.

Observable Type	Completes Automatically?	Must Unsubscribe?
HTTP (<code>http.get</code> , etc.)	Yes — after one response	No (but good practice)
<code>interval()</code>	No	Yes
<code>fromEvent()</code>	No	Yes
<code>Subject</code> / <code>BehaviorSubject</code>	Only when you call <code>.complete()</code>	Yes

The cleanest pattern for unsubscribing multiple Observables at once:

```
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

export class MyComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    // takeUntil auto-unsubscribes when destroy$ emits
    interval(1000)
      .pipe(takeUntil(this.destroy$))
      .subscribe(tick => console.log(tick));

    fromEvent(document, 'click')
      .pipe(takeUntil(this.destroy$))
      .subscribe(() => console.log('clicked'));
  }

  ngOnDestroy() {
    this.destroy$.next(); // triggers completion on all subscriptions
  }
}
```

```

    this.destroy$.complete();
  }
}

```

Output:

All Observables piped with `takeUntil(this.destroy$)` will automatically unsubscribe the moment the component is destroyed — no manual tracking of individual subscriptions needed.

9. Quick Reference

Concept / Operator	What it does	Common Use Case
<code>Observable</code>	Represents a stream of values over time	HTTP response, user events, timers
<code>Observer</code>	Defines what to do with each value	The object passed to <code>.subscribe()</code>
<code>next()</code>	Handles each emitted value	Process data as it arrives
<code>error()</code>	Handles stream errors	Show error messages, log failures
<code>complete()</code>	Handles stream completion	Hide loading spinners
Cold Observable	Starts fresh for each subscriber	HTTP requests
Hot Observable	Shared stream, no replay	Mouse events, WebSocket, Subject
<code>fromEvent(el, event)</code>	Wraps a DOM event as Observable	Click, keyup, scroll events
<code>interval(ms)</code>	Emits incrementing number every N ms	Timers, polling
<code>map(fn)</code>	Transforms each emitted value	Reshape API response

Concept / Operator	What it does	Common Use Case
<code>filter(fn)</code>	Passes only values matching condition	Filter results by category
<code>switchMap(fn)</code>	Maps to inner Observable, cancels previous	Live search / typeahead
<code>catchError(fn)</code>	Intercepts errors, returns fallback Observable	API error handling
<code>retry(n)</code>	Re-subscribes on error up to N times	Unstable network requests
<code>takeUntil(notifier\$)</code>	Completes stream when notifier emits	Auto-unsubscribe on destroy

Interview Questions

1. What is an Observable? How is it different from a Promise?
2. Explain the difference between a Cold and a Hot Observable with a real-world example.
3. What are the three callback methods of an Observer? When does each one run?
4. Why must you unsubscribe from an Observable? Which Observable types complete automatically?
5. What does `switchMap` do that `map` cannot? When would you use it?
6. What is the difference between `switchMap` and `mergeMap` ?
7. Where in the pipe should `retry` be placed relative to `catchError` ? Why?
8. What does `catchError` return? What is `of([])` doing in that context?
9. What is the `takeUntil` pattern and why is it preferred over manually tracking subscriptions?
10. How does `pipe()` work? Can you chain multiple operators? Does the order matter?

Practical Exercise

Task: Build a Live Search Component with Error Handling

Create an Angular component called `UserSearchComponent` that does the following:

1. Render an `<input>` field for the user to type a username.
2. Use `FormControl` and listen to `valueChanges`.
3. Wait **400ms** after the user stops typing before making a request (use `debounceTime`).
4. Use `switchMap` to call `https://api.github.com/search/users?q={query}` with the typed value.
5. Use `map` to extract only the `items` array from the response.
6. Display each user's `login` and `avatar_url` in a list.
7. Add `retry(1)` and `catchError` to handle failures gracefully.
8. Ensure all subscriptions are properly cleaned up on component destroy.

Expected Skills Practised: `FormControl.valueChanges`, `debounceTime`, `switchMap`, `map`, `retry`, `catchError`, `takeUntil` / `unsubscribe`.

Bonus: Add a loading spinner that shows while the request is in flight, and hide it once the response arrives or an error occurs.